

PRACTICAL-12

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define NUM_ITEMS 100000
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

int BUFFER_SIZE;

int *buffer;
int in = 0;
int out = 0;

int produced_count = 0;
int consumed_count = 0;

pthread_mutex_t buffer_mutex;
sem_t empty_slots;
sem_t full_slots;

/* Producer thread */
void *producer(void *arg)
{
    int producer_id = *(int *)arg;

    for (int i = 0; i < NUM_ITEMS / NUM_PRODUCERS; i++) {
        int item = producer_id * 1000000 + i;

        /* Wait for an empty slot */
        sem_wait(&empty_slots);

        /* Enter critical section */
        pthread_mutex_lock(&buffer_mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        produced_count++;

        pthread_mutex_unlock(&buffer_mutex);

        /* Indicate that one item is available */
        sem_post(&full_slots);
    }

    return NULL;
}

/* Consumer thread */
void *consumer(void *arg)
{
    int consumer_id = *(int *)arg;

    for (int i = 0; i < NUM_ITEMS / NUM_CONSUMERS; i++) {
        int item;

        /* Wait for an available item */
        sem_wait(&full_slots);

        /* Enter critical section */
        pthread_mutex_lock(&buffer_mutex);

        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        consumed_count++;

        pthread_mutex_unlock(&buffer_mutex);

        /* Indicate that one slot is now empty */
        sem_post(&empty_slots);
    }
}
```

```

GNU nano 7.2
        * which consumer processed the item.
        */
        if (item < 0) {
            printf("Consumer %d received an invalid item.\n",
                consumer_id);
        }
    }

    return NULL;
}

/* Run one producer-consumer experiment */
void run_experiment()
{
    pthread_t producers[NUM_PRODUCERS];
    pthread_t consumers[NUM_CONSUMERS];

    int producer_ids[NUM_PRODUCERS];
    int consumer_ids[NUM_CONSUMERS];

    struct timespec start, end;

    in = 0;
    out = 0;
    produced_count = 0;
    consumed_count = 0;

    buffer = malloc(BUFFER_SIZE * sizeof(int));

    if (buffer == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    pthread_mutex_init(&buffer_mutex, NULL);
    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);

```

```

GNU nano 7.2
        if (produced_count == NUM_ITEMS &&
            consumed_count == NUM_ITEMS) {
            printf("Synchronization check: PASSED\n");
        } else {
            printf("Synchronization check: FAILED\n");
        }

        sem_destroy(&empty_slots);
        sem_destroy(&full_slots);
        pthread_mutex_destroy(&buffer_mutex);

        free(buffer);
    }

int main()
{
    int buffer_sizes[] = {1, 10, 100, 1000};
    int number_of_tests = sizeof(buffer_sizes) /
        sizeof(buffer_sizes[0]);

    printf("Producer-Consumer Problem Using Semaphores\n");
    printf("-----\n");

    printf("Total items: %d\n", NUM_ITEMS);
    printf("Producers: %d\n", NUM_PRODUCERS);
    printf("Consumers: %d\n", NUM_CONSUMERS);

    for (int i = 0; i < number_of_tests; i++) {
        BUFFER_SIZE = buffer_sizes[i];
        run_experiment();
    }

    printf("\nAll buffer-size experiments completed.\n");

    return 0;
}

```

```

no lost updates occurred.
ruhi0404@Ruhitha:~/Operating_Systems$ nano prac12task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc -Wall -Wextra -pthread prac12task1.c -o prac12task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac12task1
Producer-Consumer Problem Using Semaphores
-----
Total items: 100000
Producers: 2
Consumers: 2

Buffer size: 1
Items produced: 100000
Items consumed: 100000
Execution time: 18.846871 seconds
Synchronization check: PASSED

Buffer size: 10
Items produced: 100000
Items consumed: 100000
Execution time: 1.691656 seconds
Synchronization check: PASSED

Buffer size: 100
Items produced: 100000
Items consumed: 100000
Execution time: 0.159387 seconds
Synchronization check: PASSED

Buffer size: 1000
Items produced: 100000
Items consumed: 100000
Execution time: 0.092765 seconds
Synchronization check: PASSED

All buffer-size experiments completed.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation:

The Producer-Consumer problem was successfully implemented using POSIX threads, counting semaphores, and a mutex-protected circular buffer. Producer threads inserted items into the shared buffer, while consumer threads removed items from it. The `empty_slots` semaphore controlled the number of available buffer positions, and the `full_slots` semaphore controlled the number of items available for consumption. The mutex protected the buffer indexes and shared counters from simultaneous access. The experiment was performed using different buffer sizes. The number of produced and consumed items remained equal for every test, indicating correct synchronization. Smaller buffers caused producers and consumers to wait more frequently, while larger buffers generally reduced waiting and improved throughput until other system overheads became dominant.

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

/* ----- DEADLOCK DEMONSTRATION ----- */

void *deadlock_thread1(void *arg)
{
    printf("Thread 1: Trying to acquire Resource 1...\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Acquired Resource 1.\n");

    sleep(1);

    printf("Thread 1: Trying to acquire Resource 2...\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Acquired Resource 2.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *deadlock_thread2(void *arg)
{
    printf("Thread 2: Trying to acquire Resource 2...\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Acquired Resource 2.\n");

    sleep(1);
```

```
GNU nano 7.2
/* ----- DEADLOCK PREVENTION ----- */

void *ordered_thread1(void *arg)
{
    printf("Thread 1: Acquiring Resource 1...\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Resource 1 acquired.\n");

    sleep(1);

    printf("Thread 1: Acquiring Resource 2...\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Resource 2 acquired.\n");

    printf("Thread 1: Using both resources.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released.\n");

    return NULL;
}

void *ordered_thread2(void *arg)
{
    /*
     * Resource ordering:
     * Always acquire Resource 1 before Resource 2.
     */
    printf("Thread 2: Acquiring Resource 1...\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 acquired.\n");

    sleep(1);

    printf("Thread 2: Acquiring Resource 2...\n");
```

```

GNU nano 7.2
/*
 * Resource ordering:
 * Always acquire Resource 1 before Resource 2.
 */
printf("Thread 2: Acquiring Resource 1...\n");
pthread_mutex_lock(&resource1);
printf("Thread 2: Resource 1 acquired.\n");

sleep(1);

printf("Thread 2: Acquiring Resource 2...\n");
pthread_mutex_lock(&resource2);
printf("Thread 2: Resource 2 acquired.\n");

printf("Thread 2: Using both resources.\n");

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

printf("Thread 2: Resources released.\n");

return NULL;
}

int main()
{
    pthread_t t1, t2;

    printf("DEADLOCK AND DEADLOCK PREVENTION\n");
    printf("=====\n");

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    /*
     * -----
     * PART 1: DEADLOCK SCENARIO

```

```

GNU nano 7.2
    */
    sleep(3);

    printf("\nDeadlock detected:\n");
    printf("Thread 1 is waiting for Resource 2.\n");
    printf("Thread 2 is waiting for Resource 1.\n");

    /*
     * Cancel the deadlocked threads so the demonstration
     * can continue to the prevention section.
     */
    pthread_cancel(t1);
    pthread_cancel(t2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    /*
     * Reinitialize mutexes after cancelling deadlocked threads.
     */
    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    /*
     * -----
     * PART 2: DEADLOCK PREVENTION
     * -----
     */

    printf("\nPART 2: DEADLOCK PREVENTION\n");
    printf("-----\n");

    printf("Resource ordering is enforced.\n");
    printf("Both threads acquire Resource 1 before Resource 2.\n\n");

```

```

printf("Resource ordering is enforced.\n");
printf("Both threads acquire Resource 1 before Resource 2.\n\n");

pthread_create(&t1, NULL, ordered_thread1, NULL);
pthread_create(&t2, NULL, ordered_thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("\nDeadlock prevention successful.\n");
printf("Both threads completed without deadlock.\n");

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

return 0;
}

```

Observation:

A deadlock scenario involving two threads and two shared resources was successfully demonstrated using POSIX mutexes. In the first part, Thread 1 acquired Resource 1 and waited for Resource 2, while Thread 2 acquired Resource 2 and waited for Resource 1. This created a circular wait and resulted in deadlock. The four necessary conditions for deadlock—mutual exclusion, hold and wait, no preemption, and circular wait—were identified in the scenario. In the second part, deadlock was prevented by enforcing a fixed resource ordering. Both threads were required to acquire Resource 1 before Resource 2. This removed the circular-wait condition, allowing both threads to complete successfully.